



ICS4U RST Step 3: Implementation



Click [HERE](#) for the Task Description

Click [HERE](#) for the RST Repository

List of websites or outside sources used:

<https://docs.oracle.com/javase/8/javafx/api/javafx/scene/control/Spinner.html> #spinner
<https://docs.oracle.com/javase/8/javafx/api/javafx/scene/layout/StackPane.html> #stackpane, allows components to be overlaid over each other. Used for the token overlays.
<https://docs.oracle.com/javase/8/docs/api/java/awt/geom/Point2D.html> #point 2d, it's just an object that stores two data points. I use this for coordinates. It's a lot cleaner of an implementation versus say tileX and tileY. That's why I use it.
<https://docs.oracle.com/javase/8/javafx/api/javafx/scene/shape/Circle.html> #circle, used to represent tokens on the monopoly board.
<https://docs.oracle.com/javafx/2/api/javafx/scene/doc-files/cssref.html>

Before submitting, verify that your code is in your [GitHub repository](#)!

Concerns <i>Areas that need improvement</i>	Criteria	Strengths <i>Areas that exceed expectations</i>	Level
A.2 Use of modular programming concepts (JavaFX) to create an interface that is user-friendly.			
	My program is intuitive, easy to use and looks professional. My interface gracefully handles both valid and invalid inputs from the user Weight: 10		
A.1, A.2, A.3 Program Concepts: object-oriented design, data structures, files, algorithms			
	My code addresses all the requirements for my program with attention to efficiency.		



ICS4U RST Step 3: Implementation



	My code demonstrates mastery of the concepts learned in units 1 to 4 Weight: 20		
B1.7 Use shared resources to manage source code effectively.			
	My GitHub repository contains multiple commits with descriptive commit messages. Weight: 5		

Code Conventions Checklist:

Criteria (Weight: 10)	No (0 pt)	Sometimes (1 pt)	Yes (2 pt)
uses whitespace and indentation as appropriate to make code easy to read			
uses proper naming conventions for packages, classes, variables, constants & methods			
uses variable & constant names that are descriptive and make code easy to understand			
validates user input (e.g. try ... catch blocks, loops, etc), and provide appropriate feedback			
limits the scope of variables and fields to the smallest necessary			
uses single-line comments (//) to document the 'why' behind coding decisions			
uses doc comments (/** */) to create internal documentation for objects that are reused			
no duplicated code (refactor into a method!) or unnecessary lines of code			
no Warnings from the Problems view or TODOs from the Task view in the Eclipse IDE			
no blocks of commented code in the final product			



ICS4U RST

Step 3: Implementation

